Spring

What is Spring?

Spring is a lightweight open-source framework based on

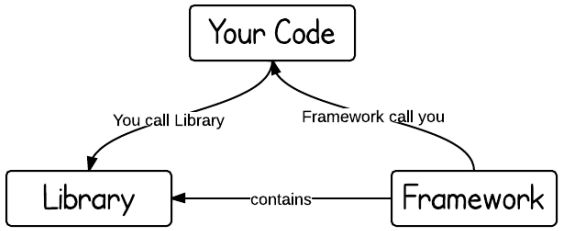
1. dependency injection (DI) and
2. aspect-oriented programming (AOP)

It is light-weighted and loosely coupled

Rod Johnson

Rod's best-selling **Expert One-on-One J2EE Design and Development (2002**) was one of the most influential books ever published on J2EE.

What is a Framework? What is a Library?



Framework: A framework is a semi complete reusable application.

It defines a **skeleton** where the application defines its own features to fill out the skeleton. Part of the coding is done for you and the remaining coding you need to do

The advantage of framework is that developers do not need to worry about basic grid or design patter of overall application

A framework usually makes use a lot of libraries to make your work easier.

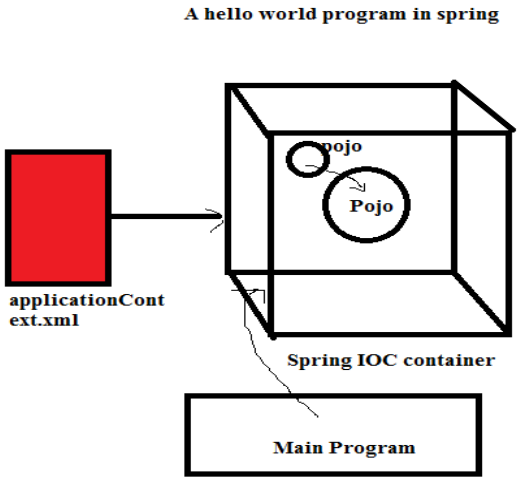
Examples of (web) frameworks are: struts, spring MVC, webwork

Library: A library is just a collection of class definitions.

You call a library function, Examples Java library is **Apache Commons**: http://commons.apache.org/

Spring Container

In a Spring application, objects are **created, are wired together, and live** in the Spring container.



There are two types of IoC containers.

1. **BeanFactory**: BeanFactory is like a factory class that contains a collection of beans. It instantiates the bean whenever asked for by clients.
2. **ApplicationContext**: The ApplicationContext interface is built on top of the BeanFactory interface. It provides some extra functionality on top BeanFactory.

Spring DI

Dependency injection is an expression coined in Martin Fowler's article

"**Inversion of Control Containers** and the Dependency Injection Pattern"

Martin Fowler

These are the **design patterns** that are used to **remove dependency from the programming code**. They make the code easier to test and maintain.

Tight Coupling Example (Without DI):

```
class Employee {
    Address address;

    Employee() {
        address = new Address();  // Dependency created inside the class
    }
}
```

Problem:
`Employee` is tightly coupled with `Address`. Any changes to `Address` require changes in `Employee`. Address object is created in the Employee class itself.

Loose Coupling Example (With DI):

```
class Employee {
    Address address;

    Employee(Address address) {  // Dependency injected through constructor
        this.address = address;
    }
}
```

Solution:
`Address` is created outside the `Employee` class and passed in, making `Employee` independent and easier to maintain.

There are two ways to inject the dependency.

1 The constructor
2 The Setter method

```
class Employee{
Address address;

//constructor injection
Employee(Address address){
this.address=address;
}
//setter injection
public void setAddress(Address address){
this.address=address;
}

}
```

What is IoC?

IoC is **a design principle in software engineering** where the program's control flow is managed by a framework or container instead of the code itself.

DI is a type of IOC

Traditional Programming vs IoC:

- **Traditional**: Our program directly controls the flow by calling libraries or frameworks → leads to **tight coupling**.
- **IoC**: The framework manages the flow and injects dependencies into your code → promotes **loose coupling**.

What is Tight Coupling?
Tight coupling means parts of the **code are highly dependent on each other**, making it hard to maintain, test, or modify.

What is Loose Coupling?
Loose coupling means that **different parts of a system** (**components**, **classes**, **modules**) have **minimal** or no **direct dependenci**es on each other. Changes in one part should have minimal or no impact on other parts.

How is Loose Coupling Achieved?

By using techniques like:

1. Dependency Injection
2. Interfaces/Abstractions
3. Inversion of Control (IoC)
4. Message Queues

Explanation: -

1. **Dependency Injection:** Injecting dependencies instead of creating them within the class.
2. **Interfaces/Abstractions:** Defining interfaces and abstract classes to decouple implementations from clients.
3. **Inversion of Control (IoC**): Handing control of object creation and dependencies to an external container (like Spring).
4. **Message Queues**: Using message queues for communication between components, allowing them to interact asynchronously**.**

Advantages/Benefits of Loose Coupling:/IOC

1. **Easier Maintenance**: Clear separation of responsibilities makes code easier to update or modify without breaking the system.
2. **Reusability**: Components can be reused in in other parts of the system
3. **Testability**: Individual components can be tested in isolation. simplifying unit testing
4. **Reduced Risk of Cascading Failures:** Changes in one part are less likely to cause unexpected failures in other parts.
5. **Flexibility**: Easier to replace or extend components
6. **Extensibility**: New functionality can be added without altering existing code, supporting scalability.

Difference between constructor and setter injection

There are many key differences between constructor injection and setter injection.

1. **Partial dependency**: can be injected using setter injection but it is not possible by constructor. Suppose there are 3 properties in a class, having 3 arg constructor and setters methods. In such case, if you want to pass information for only one property, it is possible by setter method only.
2. **Overriding**: Setter injection overrides the constructor injection. If we use both constructor and setter injection, IOC container will use the setter injection.
3. **Changes**: We can easily change the value by setter injection. It doesn't create a new bean instance always like constructor. So setter injection is flexible than constructor injection.

Difference between constructor and setter injection

Aspect	Constructor Injection	Setter Injection
Partial Injection	**Does not support partial injection**; all required dependencies must be provided.	**Supports partial injection**; dependencies can be injected selectively.
Immutability	Promotes **immutability**; dependencies cannot be modified once set.	Does not **enforce immutability**; dependencies can be changed after injection.
Property Overriding	It doesn’t override the setter property.	It overrides the constructor property
Instance Modification	Creates a **new instance if** modifications are required.	Does **not create a new instance**; updates existing dependencies.
Injection Time	Dependencies are injected **at the time of object creation.**	Dependencies are injected **after the object is created.**

Additional information:

Inversion of Control is a much more general concept and it can be expressed in many different ways. **Dependency Injection (DI) is a special case of Inversion of Control (IOC).**

1. **Inversion of Control (IoC):**IoC is a design principle in which the control of object creation and lifecycle is inverted from the application code to an external container or framework. In traditional programming, the application is responsible for creating objects and managing their lifecycle. With IoC, this responsibility is delegated to the Spring container, which manages the creation and configuration of objects.
2. **Dependency Injection (DI):** DI is a specific implementation of IoC, where the dependencies of a class are injected into the class by an external entity (typically the Spring container) rather than the class itself creating them.

   This reduces the class's coupling with its dependencies and makes the class easier to test and maintain.

Apart from 2 types of injection we also have

Field Injection (Not recommended in many cases):

The dependency is directly injected into the class fields using annotations like @Autowired (Spring) or similar.

```
class Employee {
    @Autowired
    Address address; // Injected directly into the field
}
```

Note: Field injection is less preferred **because it makes testing harder and hides dependencies.**

Which One to Use?

- Use **Constructor Injection** for mandatory dependencies (best practice).
- Use **Setter Injection** for optional dependencies.
- Avoid **Field Injection** unless necessary (e.g., in frameworks that enforce it).